Leader
Local leader
See :help <leader>
See :help <localleader>
A special string used to start a map, instead of using a special combination.
Local leader is a leader specific to a buffer.

e.g. A useful leader is **<space>**.

```
" Set space as leader
let mapleader ="\<space>"
map <leader>s :w<cr>
imap <leader>s :w<cr>
```

Commands

```
:help map.txt
:help :map       " map command
:help

"" map
" List all mapping
:map


"" map LH RH
" Map key sequence of commands LH to RH.
" RH is same as if it was typed.
:map <leader>c :echo 'Hello'<cr>      " Same as
typing :echo 'Hello' and hitting enter
:map <leader>k <esc>                  " Same as typing <esc>


"" map <expr> LH EXPR
" Dynamically obtains the right hand sequence.
" EXPR is evaluated when the map is triggered, not when it is
defined.
:map <expr> <leader>c x >
y ? ":echo 'Hello'<cr>" : "echo 'Nothing'<cr>"


:noremap                        No recursive mapping
:noiremap

:unmap <C-G>                    Remove the map

:nunmap <C-G>                   nmap
:iunmap <C-G>                   imap
:mapclear
```

Modes

:map Normal mode maps.
:imap Insert mode maps
:vmap Visual and select modes
:xmap Visual mode
:smap Select mode
:cmap Commandline mode
:omap

Key codes

```
" Helps
:help key-codes
:help index               Default maps

<leader>
<BS>            Backspace
<CR>            Carriage return
<Enter>         Same as <CR>
<Space>
<Bar>           |
<Bslash>        \
<Del>           Delete key

<Up>, <Down>, <Left>, <Right>
<F1> ~ <F12>
<C-A>           Control-A
<S-A>           Shift-A
<M-A>           Meta-A (alt/option on Mac)
<A-A>           Same as <M-A>
<D-A>           Command-A (Mac only)
```

Default mappings

```
"
" Commandline mode (when pressed
"
<C-C>                   Exit command line mode.

<C-W>                   Delete word before cursor
<C-U>                   Delete to from beginning to cursor.

<C-R>%                  Insert current file name
<C-R>/                  Last search
<C-R>:                  Last command
<C-R>=1+1                    2. expression
<C-R>=expand('<cword>')      word under cursor.
<C-R><C-W>              Insert word under cursor
<C-R><C-A>              Insert WORD under cursor
```

Examples

```
"
map <leader>t :echo "a"<cr>                    " command
map <leader>t :echo "a"<cr>:echo "b"<cr>       " multiple commands
(without bar)
map <leader>t :echo "a"<cr>\|:echo "b"<cr>     " multiple with
bar.
" without |
" map <leader>t :echo "a"<cr>|:echo "b"<cr>
a
b
" with |
" map <leader>t :echo "a"<cr>|:echo "b"<cr>
a
<clear window>
b

map <leader>t :if input('Quit all? (y/n) ') == 'y' \| quitall \|
endif<cr>        " prompt
map <leader>t :call input('Test')            " function

" use other maps
map <leader>t j             " j=down one line map
map <leader>t 5j            " 5j
map <leader>t ihello        " i=insert mode + write "hello"
```

```
map <leader>t ihello<ESC>  " i=insert mode + write "hello" + exit
insert mode
```

```
 ============================================================
                                ======
```

Tricks

Get word under cursor

See
  :help expand()
  :help cmdline-special <cword> and other special values
Use expand("<cword>")

```
:map <c-c> /<c-r>=expand("<cword>")<cr>        Searches with the
word under the cursor.
:map <c-c>r :%s<c-r>=expand("<cword>")<cs>     Prepares to replace
the word under the cursor.
```

Use sequence of expressions with |

See
:help :bar bar help
:help map-bar mapping bar
| can be used but needs to be escaped.
Otherwise, it becomes | between the map command and the keys, instead of | between the keys.

```
" GOOD
:map <leader>c :if x>y \| echo 'Greater' \| endif<cr>

" BAD | is not escaped
:map <leader>c :if x>y | echo 'Greater' | endif<cr>
-------------------------------------------------------------------
----
" Same as below, which is nonsense.
:map <leader> :if x>y
echo 'Greater'
endif <cr>
```

Prompt for input

Use ":if input()"

```
" e.g. Prompt for y/n
" Quits only if user enters "y"
map <leader>q :if input('Quit all? (y/n) ') == 'y' \| quitall \|
endif<cr>
```

Use conditional expression in map

Use <expr> mapping to dynamically obtain the sequence

```
:map <expr> <leader>c x > y ? ':echo "x is
greater"<cr>' : ':echo "y is greater"<cr>'
```